

EOS

Agenda

Documentação Técnica

Programação III — UNOESC *campus* Videira

Daniella V. Schmidt, Yuliangel Herrera, Leandra de Oliveira

Prof. Wagner Titon

2026

SUMÁRIO

1. Visão Geral.....	2
1.1 Problema que o sistema resolve.....	2
2. Tecnologias Utilizadas.....	2
3. Arquitetura do Sistema.....	3
3.1 O que é MVC?.....	3
3.2 Camadas extras.....	4
Service (Serviço).....	4
Factory (Repositório).....	4
DTO (Data Transfer Object).....	4
3.3 Estrutura total.....	5
4. Design Patterns (Padrões de Projeto).....	6
4.1 Factory Pattern.....	6
4.2 Service Layer (Camada de Serviço).....	6
4.3 Data Transfer Object (DTO) Pattern.....	6
5. Banco de Dados.....	7
5.1 Tabelas e seus campos.....	7
5.3 Exemplo de migration.....	8
6. DIAGRAMAS.....	8
7. PRINTS DA APLICAÇÃO.....	11
8. API REST.....	13
8.1 Rotas disponíveis.....	14
8.2 Exemplo de resposta JSON.....	14
8.3 Tratamento de erros.....	14
9. Dificuldades Encontradas.....	15

1. Visão Geral

O EOS é um sistema de agenda desenvolvido para auxiliar usuários na organização de compromissos, tarefas e eventos do cotidiano de forma prática, intuitiva e eficiente. Seu nome foi inspirado em Eos, a deusa grega da aurora, responsável por anunciar o início de um novo dia. A escolha desse nome representa a proposta central do sistema: oferecer ao usuário uma visão clara de suas atividades futuras, permitindo que ele planeje melhor seu tempo e tome decisões de forma mais organizada.

Assim como a deusa Eos traz a luz que antecede o nascer do sol, o sistema busca proporcionar clareza e previsibilidade sobre os compromissos do usuário, ajudando-o a visualizar sua rotina antes mesmo que ela aconteça. Dessa forma, o EOS não atua apenas como uma agenda digital, mas também como uma ferramenta de apoio ao planejamento pessoal e profissional.

O principal objetivo do sistema é permitir que qualquer pessoa gerencie seus compromissos de maneira simples, visual e inteligente, reduzindo esquecimentos, conflitos de horários e dificuldades relacionadas à administração do tempo.

Nome: EOS, derivado do termo grego "Ἠώς" (Aurora). Na mitologia grega, Eos era a deusa responsável por abrir as portas do céu todas as manhãs para a passagem do carro do Sol. Seu simbolismo está diretamente relacionado a novos começos, planejamento e organização, conceitos que se alinham à proposta do sistema.

1.1 Problema que o sistema resolve

Embora existam diversas ferramentas digitais disponíveis atualmente, muitas pessoas ainda utilizam métodos tradicionais para controlar seus compromissos, como cadernos, agendas físicas, anotações em blocos de papel ou lembretes simples no celular. Apesar de funcionarem para registros básicos, essas soluções apresentam limitações importantes quando o objetivo é manter uma rotina organizada.

Entre os principais problemas encontrados nesses métodos estão a ausência de notificações automáticas, a dificuldade de visualizar a distribuição dos compromissos ao longo da semana e a impossibilidade de identificar rapidamente conflitos de horário ou excesso de atividades em um mesmo dia.

Além disso, quando os compromissos aumentam, torna-se mais difícil perceber períodos livres na agenda ou antecipar situações de sobrecarga, o que pode resultar em atrasos, esquecimentos e queda na produtividade.

O EOS foi desenvolvido justamente para solucionar essas dificuldades por meio de recursos inteligentes que tornam a gestão do tempo mais eficiente. Entre suas principais funcionalidades destacam-se:

- Calendário visual e interativo para visualização clara dos compromissos;
- Cadastro e gerenciamento de eventos de forma simples e rápida;
- Alertas que identificam dias com excesso de atividades agendadas;
- Lembretes automáticos enviados antes de cada compromisso;
- Organização por categorias personalizadas com cores distintas;
- Sugestões de horários livres para melhor distribuição das tarefas;
- Facilidade de uso mesmo para usuários com pouca experiência tecnológica.

2. Tecnologias Utilizadas

As tecnologias utilizadas no desenvolvimento do EOS foram selecionadas considerando critérios como estabilidade, desempenho, facilidade de manutenção e ampla adoção no mercado de desenvolvimento de software. Outro fator importante foi a compatibilidade com o ambiente utilizado pela equipe, baseado no XAMPP e no sistema operacional Windows.

Além disso, buscou-se utilizar ferramentas gratuitas e de código aberto, permitindo que o projeto pudesse ser executado sem custos adicionais de licenciamento. Todo o sistema funciona diretamente em um ambiente PHP tradicional.

Camada	Tecnologia	Versão	Para que serve
Back-end	Laravel	13.12.0	Framework responsável pela estrutura geral da aplicação, gerenciamento de rotas, autenticação, acesso ao banco de dados e organização do código.
Linguagem	PHP	8.4.10	Linguagem utilizada para implementar toda a lógica de negócio do sistema.
Banco de dados	SQLite	nativa	Responsável pelo armazenamento persistente das informações da aplicação.
Template (dinâmico)	Blade (Laravel)	nativa	Motor de templates utilizado para gerar as páginas HTML de forma dinâmica.

3. Arquitetura do Sistema

O EOS foi desenvolvido utilizando a arquitetura MVC (Model-View-Controller), complementada pelas camadas de Service, Repository e DTO (Data Transfer Object). Essa estrutura foi adotada tanto para atender aos requisitos do projeto quanto para seguir padrões amplamente utilizados na indústria de software.

A principal vantagem dessa abordagem é a separação clara de responsabilidades. Cada camada possui uma função específica dentro do sistema, evitando acoplamento excessivo e facilitando a manutenção, testes e evolução do código ao longo do tempo.

Em vez de concentrar toda a lógica em um único local, cada componente executa apenas as tarefas que lhe competem, tornando a aplicação mais organizada e compreensível para futuros desenvolvedores.

3.1 O que é MVC?

MVC é um padrão arquitetural que divide a aplicação em três componentes principais: Modelo (Model), Visão (View) e Controlador (Controller). Essa separação permite que a interface, a lógica de negócio e os dados sejam tratados de forma independente.

Camada	O que faz	Exemplo
Model (Modelo)	Representa os dados e a estrutura do banco de dados. Ela é responsável por armazenar, recuperar e manipular informações persistentes.	<code>`app/Models/Event.php`</code> Event representa um compromisso cadastrado na agenda.

View (Visão)	É o que o usuário vê na tela: HTML, formulários, calendário. Ela define como as informações serão exibidas na tela e permite a interação entre usuário e sistema.	<code>`resources/views/events/index.blade.php`</code> Essa página pode exibir o calendário, formulários de cadastro e listas de compromissos.
Controller (Controlador)	Atua como intermediário entre a interface e a lógica da aplicação. Ele recebe as requisições do usuário, encaminha o processamento para os serviços adequados e retorna as respostas necessárias.	<code>`app/Http/Controllers/EventController.php`</code> Gerencia ações relacionadas aos eventos, como cadastro, edição, exclusão e consulta.

3.2 Camadas extras

Embora o padrão MVC já ofereça uma boa organização estrutural, aplicações mais complexas tendem a exigir camadas adicionais para evitar que responsabilidades diferentes fiquem concentradas em um único componente.

Service (Serviço)

A camada Service concentra toda a lógica de negócio do sistema. Ela contém as regras que determinam como o EOS deve se comportar em diferentes situações.

Dessa forma, os Controllers permanecem simples, atuando apenas como intermediários entre a interface e os serviços.

Exemplos:

- ``Event Service.php``: responsável por validar conflitos de horários, verificar sobrecarga de compromissos e gerenciar regras relacionadas aos eventos.
- ``Suggestion Service.php``: analisa padrões de utilização e sugere horários livres para novos compromissos.

Factory (Repositório)

O padrão Factory foi utilizado para centralizar a criação de objetos e entidades, especialmente no contexto de testes e sementeira do banco de dados. Em vez de instanciar modelos manualmente com dados fixos, utilizamos factories para definir valores padrão e gerar registros de forma consistente e controlada.

Exemplos:

- `UserFactory.php`: define atributos padrão para criação de usuários (nome, e-mail, senha, timezone, etc.).

DTO (Data Transfer Object)

Os DTOs são objetos utilizados para transportar dados entre diferentes camadas do sistema de forma estruturada e segura.

Sua principal função é evitar que os Models sejam expostos diretamente durante a comunicação entre componentes, reduzindo dependências e melhorando a organização do código.

Exemplo:

- ``EventDTO.php``: encapsula informações de um evento, como título, data, horário e categoria, permitindo que esses dados sejam transmitidos entre as camadas da aplicação de maneira padronizada.

A utilização conjunta de MVC, Services, Repositories e DTOs torna a arquitetura do EOS mais robusta, organizada e preparada para futuras expansões, seguindo práticas amplamente adotadas no desenvolvimento profissional de software.

3.3 Estrutura total

A seguir, a estrutura completa de diretórios do projeto:

```
smart-agenda/
├── app
│   ├── DTO
│   │   ├── Auth
│   │   ├── Calendar
│   │   └── Event
│   ├── Http
│   │   ├── Controllers
│   │   │   └── Auth
│   │   ├── Middleware
│   │   └── Requests
│   │       └── Auth
│   ├── Models
│   ├── Providers
│   ├── Repositories
│   ├── Services
│   ├── View
│   │   └── Components
│   └── config
├── database
│   ├── factories
│   ├── migrations
│   └── seeders
├── public
│   ├── build
│   │   └── assets
│   ├── css
│   ├── imgs
│   │   ├── logo
│   │   └── web-development
│   └── js
├── resources
│   ├── css
│   ├── js
│   └── views
│       ├── auth
│       ├── calendars
│       ├── components
│       ├── contacts
│       ├── dashboard
│       ├── events
│       ├── layouts
│       ├── preferences
│       ├── profile
│       └── partials
├── routes
├── storage
│   ├── app
│   │   ├── private
│   │   └── public
│   ├── framework
│   │   ├── cache
│   │   │   └── data
│   │   ├── sessions
│   │   ├── testing
│   │   └── views
│   └── logs
```

4. Design Patterns (Padrões de Projeto)

Os Design Patterns, ou padrões de projeto, são soluções amplamente utilizadas para resolver problemas recorrentes no desenvolvimento de software. Em vez de fornecer código pronto, eles estabelecem formas de organizar a aplicação de maneira mais estruturada, facilitando sua manutenção, escalabilidade e evolução ao longo do tempo. No desenvolvimento foram adotados os padrões Repository, Service Layer e DTO, cada um desempenhando um papel importante na organização.

4.1 Factory Pattern

No projeto:

- ``database/factories/UserFactory.php``

No projeto, o padrão Factory foi utilizado para centralizar a criação de objetos complexos, especialmente aqueles relacionados a modelos e entidades do banco de dados. Em vez de instanciar objetos diretamente com `new`, utilizamos factories para definir a estrutura e os dados padrão, facilitando a criação de registros de teste, a sementeira do banco e a padronização da criação de instâncias.

No EOS, as factories atuam como uma camada que abstrai a lógica de construção de objetos, garantindo que modelos como User sejam criados com atributos consistentes e válidos. Isso torna o código mais limpo, reduz a duplicação e facilita a manutenção, especialmente em cenários de testes e seeding.

4.2 Service Layer (Camada de Serviço)

No projeto:

- ``app/Services/EventService.php``
- ``app/Services/SuggestionService.php``

A camada de serviços é responsável por concentrar toda a lógica de negócio do sistema. Enquanto os Controllers recebem as requisições dos usuários e retornam as respostas apropriadas, os Services executam as regras necessárias para o funcionamento da aplicação.

No EOS, funcionalidades como validação de conflitos de horário, análise de sobrecarga de compromissos e sugestões de horários livres são processadas nessa camada. Essa separação evita que os Controllers se tornem excessivamente complexos e permite que as regras de negócio sejam reutilizadas em diferentes partes do sistema, contribuindo para uma estrutura mais limpa e organizada.

4.3 Data Transfer Object (DTO) Pattern

No projeto:

- ``app/DTO/Event/EventDTO.php``
- ``app/DTO/Calendar/CalendarDTO.php``
- ``app/DTO/Auth/LoginDTO.php``

O padrão DTO foi adotado para realizar o transporte de dados entre as diferentes camadas da aplicação de forma controlada e padronizada. Esses objetos possuem apenas os atributos necessários para a transferência das informações, sem conter regras de negócio ou dependências relacionadas ao banco de dados.

No contexto do EOS, os DTOs permitem que apenas os dados relevantes sejam compartilhados entre Controllers, Services e Repositories, evitando a exposição direta dos Models da aplicação. Como resultado, o sistema apresenta menor acoplamento entre

componentes, maior segurança na manipulação das informações e uma estrutura mais adequada para manutenção e testes futuros

A utilização conjunta desses padrões contribui para uma arquitetura mais robusta, organizada e alinhada às boas práticas de desenvolvimento de software. Além de facilitar a manutenção do código, essa abordagem torna o sistema mais escalável e preparado para futuras expansões e melhorias.

5. Banco de Dados

O banco de dados do EOS usa SQLite é criado automaticamente pelo sistema de migrations do Laravel. Isso significa que não é preciso entrar no phpMyAdmin e criar as tabelas manualmente — basta rodar um comando e o Laravel cria tudo.

Comando para criar o banco: `php artisan migrate` Comando para popular com dados de teste: `php artisan db:seed`

5.1 Tabelas e seus campos

A estrutura do banco de dados do projeto foi definida para atender ao fluxo mínimo de uma Agenda Inteligente, permitindo o cadastro de usuários, eventos, preferências, contatos, lembretes, participantes e solicitações inteligentes. O modelo foi pensado para ser simples o suficiente para um MVP, mas flexível para futuras evoluções, como múltiplos calendários, integração com serviços externos e sugestões automáticas de horários.

A tabela ``users`` representa os usuários do sistema. Ela armazena informações básicas como identificador, nome, e-mail, senha, fuso horário e datas de criação e atualização. Essa tabela é a base do relacionamento com praticamente todas as demais entidades.

A tabela ``calendars`` representa a agenda do usuário. Embora no MVP exista apenas uma agenda principal por usuário, essa tabela foi mantida para permitir expansão futura, como agenda pessoal, profissional ou acadêmica.

A tabela ``events`` armazena os compromissos cadastrados na agenda. Essa é uma das principais tabelas do sistema, pois centraliza os eventos criados manualmente ou a partir de solicitações inteligentes.

A tabela ``smart_requests`` registra os pedidos feitos pelo usuário em linguagem natural, como “marque uma reunião amanhã às 15h”. Ela armazena o texto original no campo `rawText`, o tipo de intenção, os dados extraídos pela inteligência, além do status.

A tabela ``event_suggestions`` guarda sugestões de horários geradas pelo sistema quando existe conflito ou necessidade de sugerir alternativas. Essa tabela permite que o sistema apresente opções ao usuário antes da criação definitiva do evento.

A tabela ``user_preferences`` armazena as preferências individuais do usuário, utilizadas para personalizar o comportamento da agenda.

A tabela `event\_reminders` representa os lembretes vinculados a um evento. Com ela, o sistema consegue controlar notificações antes do início dos compromissos.

A tabela `event\_participants` armazena os participantes de cada evento. Ela permite associar contatos cadastrados aos eventos ou adicionar participantes manualmente.

A tabela `contacts` representa a lista de contatos do usuário. Essa tabela facilita a criação de eventos com participantes já conhecidos pelo sistema.

Em conjunto, essas tabelas formam a base necessária para o funcionamento do MVP da Agenda Inteligente, permitindo criar eventos, interpretar solicitações em linguagem natural, sugerir horários, armazenar preferências do usuário e gerenciar contatos, participantes e lembretes.

5.3 Exemplo de migration

No Laravel, as tabelas são criadas por migrations — arquivos PHP que descrevem a estrutura do banco. Exemplo da migration de events:

```
Schema::create("events", function (Blueprint $table) {
    $table->id();
    $table->foreignId("user_id")->constrained()->onDelete("cascade");

    $table->foreignId("category_id")->nullable()->constrained()->nullOnDelete();
    $table->string("title", 150);
    $table->text("description")->nullable();
    $table->dateTime("start_datetime");
    $table->dateTime("end_datetime");
    $table->string("location", 200)->nullable();
    $table->string("color", 7)->default("#3788d8");
    $table->boolean("is_recurring")->default(false);
    $table->string("recurrence_rule")->nullable();
    $table->timestamps();
});
```

6. DIAGRAMAS

Diagrama do inicial da estruturação do banco de dados

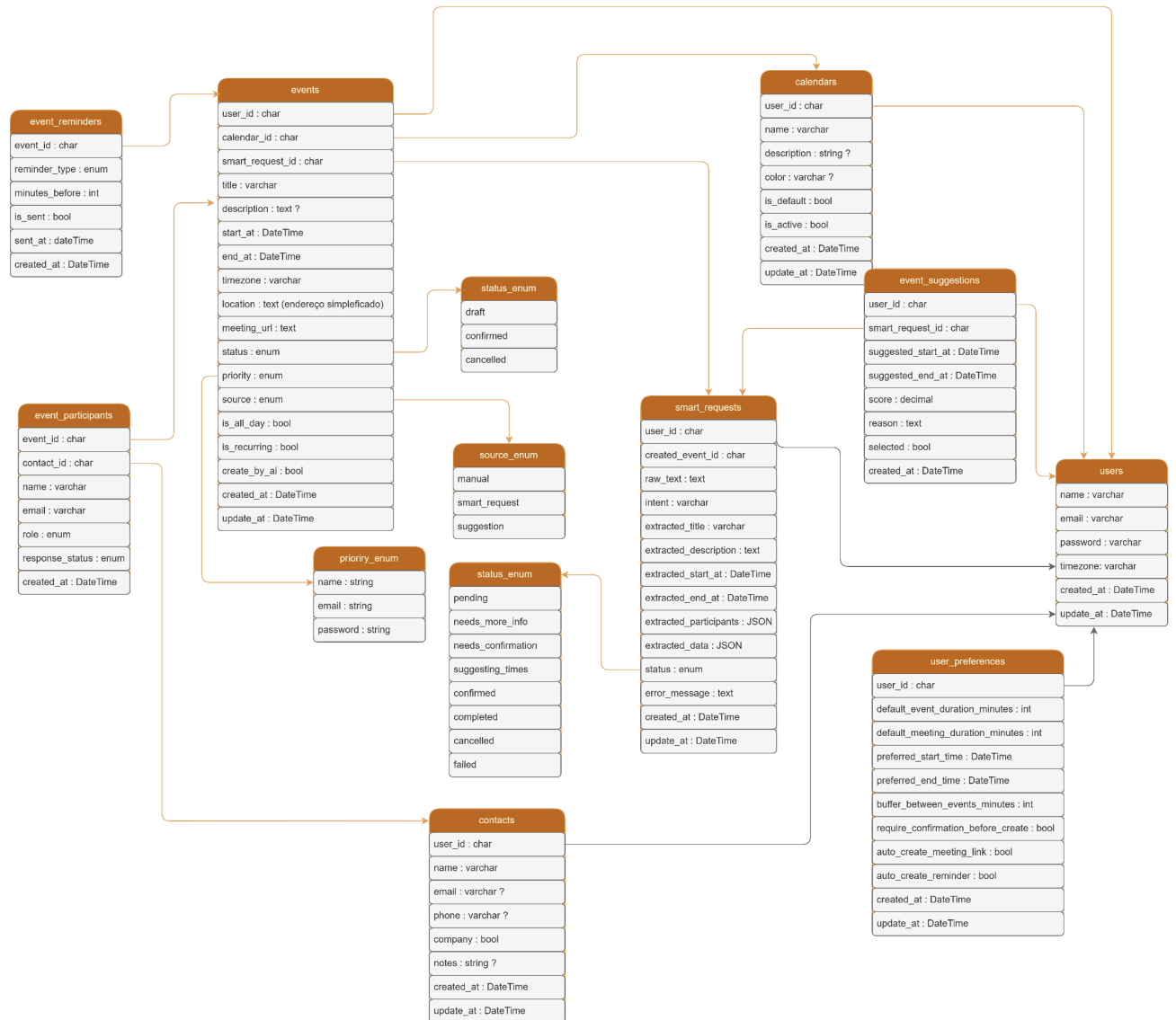


Diagrama de sequência

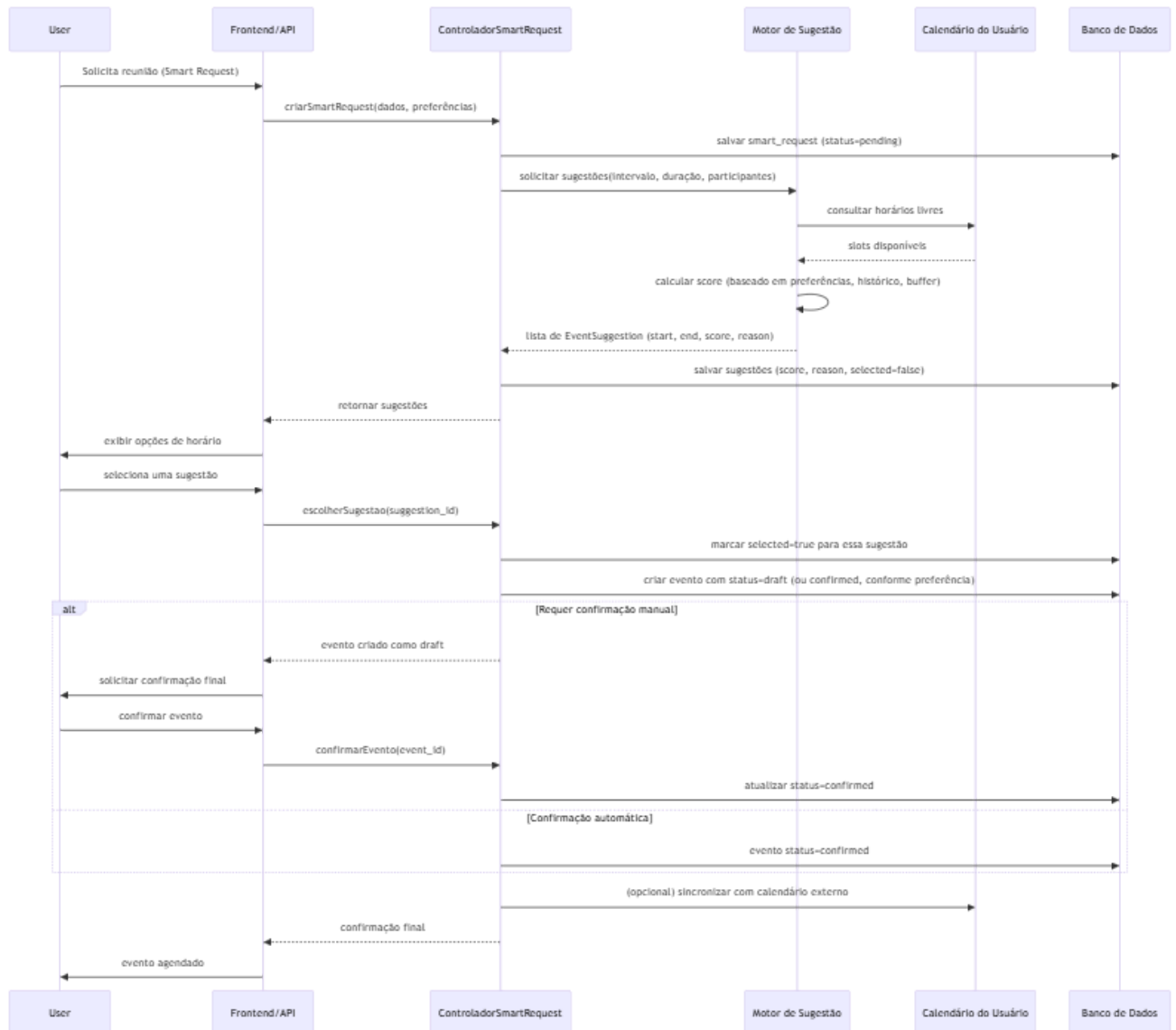
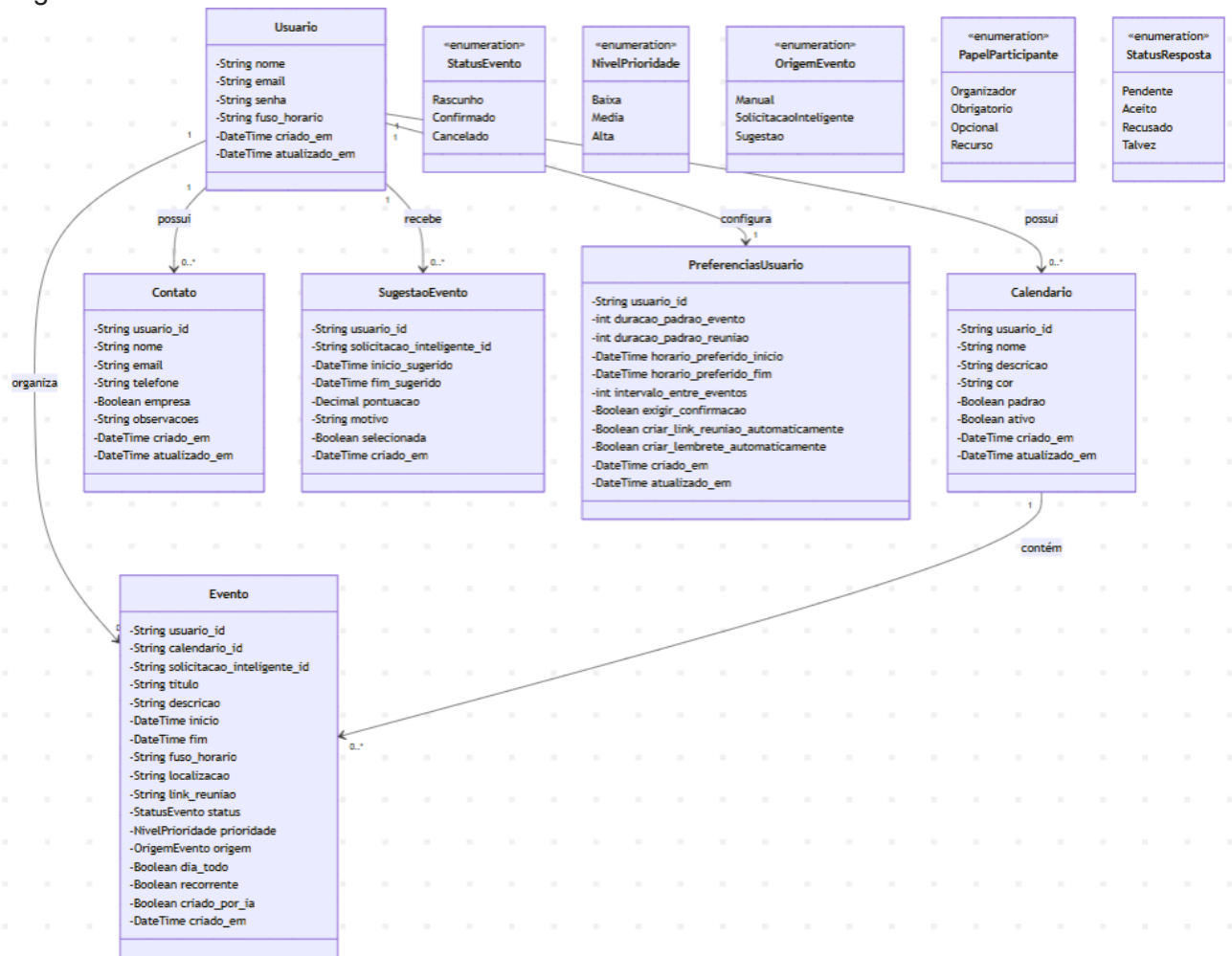


Diagrama de classe



7. PRINTS DA APLICAÇÃO

A seguir prints de alguma views presentes na aplicação da agenda:



EOS Agenda

Home

Agenda

Inteligência

José Pereira

MENU

Agenda

Minha agenda

Solicitações inteligentes

Contatos

Preferências

Configurações

CALENDÁRIOS

Meu Calendário

Trabalho

Fitness

Pessoal

Visualização

Junho de 2026

Dia

Semana

Mês

Lista

SEG	TER	QUA	QUI	SEX	SÁB	DOM
01/06	02/06	03/06	04/06	05/06	06/06	07/06
Sem eventos.	Sem eventos.	1 evento(s)	Sem eventos.	Sem eventos.	Sem eventos.	1 evento(s)
08/06	09/06	10/06	11/06	12/06	13/06	14/06
Sem eventos.	Sem eventos.	Sem eventos.	Sem eventos.	Sem eventos.	Sem eventos.	1 evento(s)
15/06	16/06	17/06	18/06	19/06	20/06	21/06
Sem eventos.	3 evento(s)	Sem eventos.	Sem eventos.	Sem eventos.	1 evento(s)	1 evento(s)
22/06	23/06	24/06	25/06	26/06	27/06	28/06
1 evento(s)	Sem eventos.	1 evento(s)	1 evento(s)	1 evento(s)	Sem eventos.	1 evento(s)
29/06	30/06	01/07	02/07	03/07	04/07	05/07
Sem eventos.	1 evento(s)	Sem eventos.	Sem eventos.	Sem eventos.	Sem eventos.	Sem eventos.

EOS Agenda

Home

Agenda

Inteligência

José Pereira

SOLICITAÇÕES INTELIGENTES

Crie eventos usando linguagem natural

NOVO PEDIDO

O que voce quer agendar?

Dentista sexta às 10h

Call com cliente na quinta, 15h, 45 min

Revisão do projeto hoje 18h com Ana e Pedro

Academia todo dia às 7h por 1h

Almoço com equipe amanhã ao meio-dia

Apresentação para o cliente na segunda às 9h, 2 horas

Ex: Marque uma reunião com Joao amanha as 15h por uma hora

Enviar pedido

0/1000

RECENTES

Solicitações

Academia todo dia às 7h por 1h

REVISÃO

PEDIDO ORIGINAL

Workshop de design thinking para equipe de inovação dia 28/06 às 9h, duração 3h

STATUS

Evento criado

INTENÇÃO

create_event

TÍTULO

Workshop Design Thinking

DESCRIÇÃO

Capacitação da equipe de inovação em metodologias ágeis

INÍCIO

06/06/2026 09:00

FIM

18/06/2026 12:00

PARTICIPANTES

equipe.inovacao@empresa.com

Salvar revisão

Confirmar evento

Excluir

Ver na agenda

EOS Agenda

Home

Agenda

Inteligência

José Pereira

Preferências

Salvar alterações

Configure seus horários, eventos, lembretes e comportamento da agenda inteligente.

Essas configurações serão usadas como padrão ao criar eventos manualmente ou por solicitação inteligente.

EVENTOS

Duração padrão

Defina os padrões usados na criação de eventos e reuniões.

Evento comum

minutos

Reunião

minutos

HORÁRIOS

Horário preferido

A agenda usará esse intervalo para sugerir horários e evitar compromissos fora do seu período ideal.

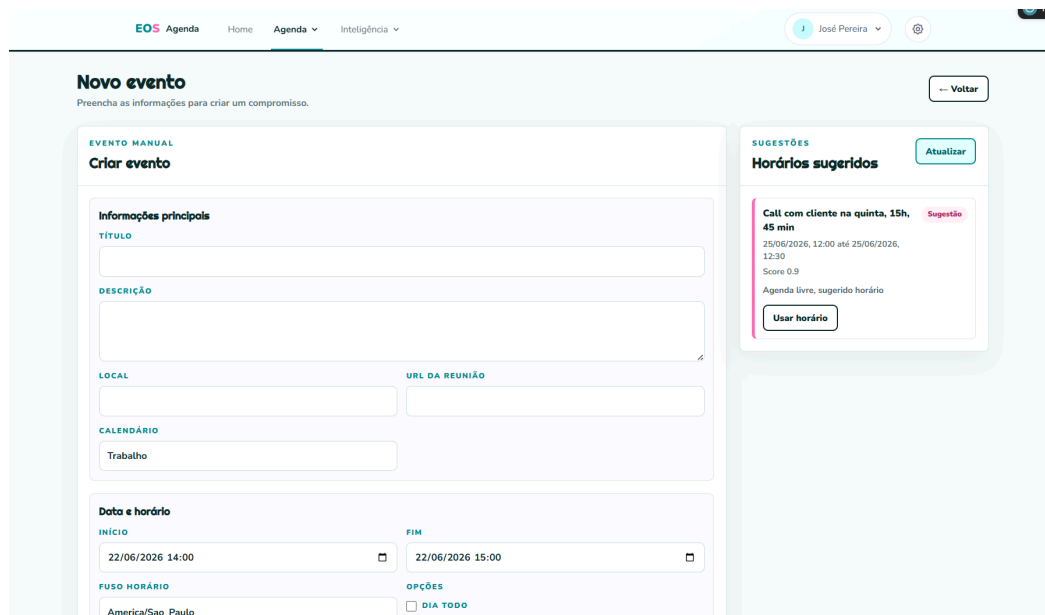
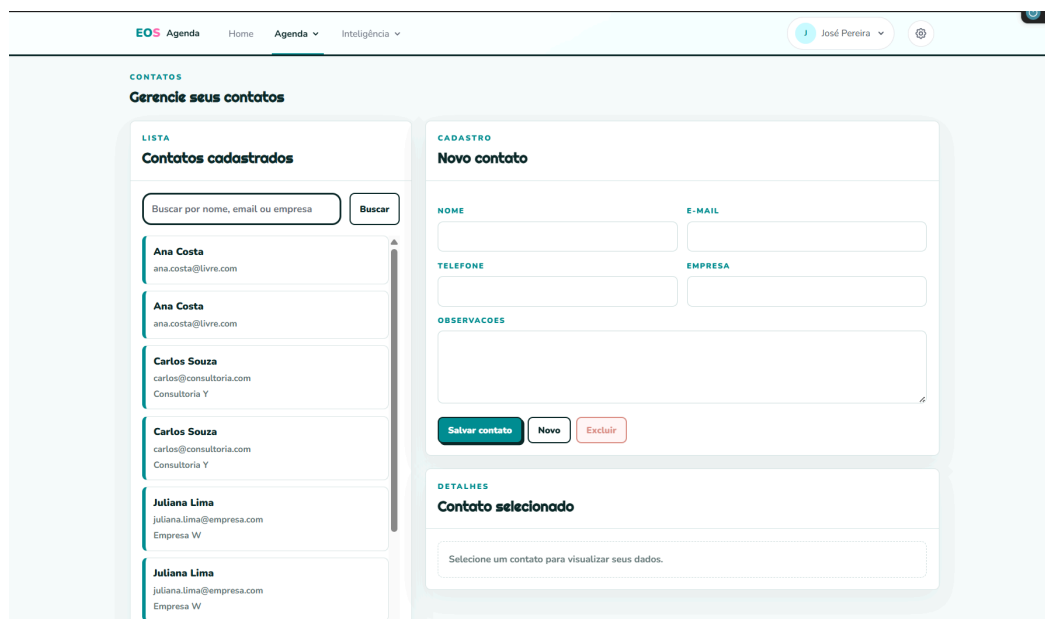
Início

Fim

AGENDA

Organização da agenda

Esse tempo será considerado ao sugerir horários livres entre compromissos.



PS.: Existem mais views, mas foi colocado apenas esses screenshots para que o documento não ficasse maior do que já está.

8. API REST

O EOS conta com uma API REST completa. Isso significa que o front-end pode consumir dados do back-end via requisições JavaScript (fetch), e que o sistema poderia ser integrado a um aplicativo mobile no futuro.

As rotas da API ficam no arquivo routes/web.php e retornam sempre dados em formato JSON.

8.1 Rotas disponíveis

Método HTTP	Rota	O que faz
GET	/api/events	Lista todos os eventos do usuário logado.
POST	/api/events	Cria um novo evento.
GET	/api/events/{id}	Retorna os dados de um evento específico.
PUT	/api/events/{id}	Atualiza um evento existente.
DELETE	/api/events/{id}	Deleta um evento.
GET	/api/categories	Lista as categorias do usuário.
GET	/api/suggestions/best-time	Retorna sugestões de horários livres.

8.2 Exemplo de resposta JSON

Quando o front-end chama GET /api/events, o retorno tem este formato:

```
{
  "data": [
    {
      "id": 1,
      "title": "Reunião com cliente",
      "start_datetime": "2026-06-10T14:00:00",
      "end_datetime": "2026-06-10T15:00:00",
      "color": "#3788d8",
      "category": {
        "id": 2,
        "name": "Trabalho",
        "color": "#F4A261"
      }
    }
  ]
}
```

8.3 Tratamento de erros

A API responde com códigos HTTP adequados para cada situação:

Código HTTP	Significado	Quando acontece no EOS
200	OK	Requisição bem-sucedida (lista, exibição).

201	Created	Evento criado com sucesso.
422	Unprocessable Entity	Dados inválidos no formulário (ex: data em branco).
404	Not Found	Evento não encontrado ou não pertence ao usuário.
401	Unauthorized	Usuário não autenticado.
500	Server Error	Erro interno inesperado no servidor.

9. Dificuldades Encontradas

Dificuldade	Descrição
Tempo de desenvolvimento	Projeto de grande porte com prazo reduzido, inviabilizando a conclusão de todas as funcionalidades planejadas.
Documentação	Falta de hábito e de processos estabelecidos para documentar adequadamente o sistema, gerando retrabalho e dificuldades de manutenção.
Utilização de novo framework	Adoção do Laravel (framework com o qual a equipe não tinha experiência prévia), exigindo curva de aprendizado e impactando a produtividade inicial.
Decisões de design pattern	Mudanças na arquitetura durante o desenvolvimento, com alterações na escolha e aplicação de padrões de projeto, causando refatorações e instabilidade.
Ambiente e Infraestrutura	Desafios na configuração e sincronização dos ambientes de desenvolvimento local e produção, resultando em falhas de compatibilidade.
Estratégia e Automação de Testes	Ausência de uma rotina automatizada de testes que aumentou a incidência de bugs de regressão a cada nova funcionalidade implementada.
Gerenciamento de Dependências	Conflitos entre pacotes e bibliotecas durante a evolução do projeto, exigindo refatorações inesperadas para garantir a compatibilidade do ecossistema.